REPUBLIQUE TUNISIENNE
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
Université de Carthage
Institut National des Sciences Appliquées et de Technologie

Rapport de Travaux Pratiques

Module : DevOps

Filière : Réseaux Informatiques et Télécommunications (RT5) – Groupe 1

TP2 : Kubernetes – Principes de base

Partie B : Mise en pratique avec kind

Réalisé par

Hanine BEN AMOR
Farah AYEB
Emna DRIHEM
Yasmine FERJANI

Encadré par : **Mme Saloua BEN YAHIA**

Année Universitaire : 2026 / 2027

Table des matières

Introduction générale **1**

1 Mise en place de l'environnement **2**

1.1 Installation de kind 2

1.2 Création du cluster 3

1.3 Vérification 4

2 Déploiement progressif **5**

2.1 Le Pod 5

2.2 Le Deployment 7

3 Le Networking avec les Services **9**

3.1 ClusterIP 9

3.2 NodePort 10

3.3 LoadBalancer 11

3.4 Exercice : l'équilibrage de charge 12

4 Stratégies de déploiement **14**

4.1 Rolling Update (par défaut) 14

4.2 Stratégie Recreate 15

4.3 Simulation de Canary Release 18

5 Application multi-conteneurs avec ClusterIP et Ingress **20**

5.1 Contrôleur Ingress 20

5.2 Application multi-conteneurs 21

6 Questions de validation **26**

6.1 Troubleshooting : ImagePullBackOff 26
6.2 Réseau : accès depuis le navigateur via localhost 27
6.3 Clean-up 27

Conclusion générale **28**

Introduction générale

Dans cette partie, on reproduit les manipulations de base de Kubernetes sur un cluster **kind** (*Kubernetes in Docker*) à 3 nœuds : pods, déploiements, services, stratégies de déploiement et Ingress. Comme pour la partie A, on travaille sous **Windows 11 + WSL 2 (Ubuntu)** avec **Docker Desktop** ; chaque nœud kind est un conteneur Docker. Les fichiers de cette partie se trouvent dans `~/TP2_Devops_K8s/PartB`.

1

Mise en place de l'environnement

1.1 Installation de kind

kubectl était déjà installé (partie A). On installe kind v0.33.0 dans WSL à partir du binaire officiel. Pour libérer de la mémoire, on arrête aussi le cluster Minikube de la partie A (`minikube stop`, sans rien supprimer).

```
curl -Lo ./kind https://kind.sigs.k8s.io/dl/v0.33.0/kind-linux-amd64
sudo install ~/kind /usr/local/bin/kind && rm ~/kind
kind version
```

```
user@DESKTOP-QGV8NUU:~$ sudo install ~/kind /usr/local/bin/kind && rm ~/kind

kind version
[sudo] password for user:
kind v0.33.0 go1.26.7 linux/amd64
```

FIGURE 1.1 – Installation de kind dans `/usr/local/bin` puis `kind version` : `kind v0.33.0 go1.26.7 linux/amd64`.

1.2 Création du cluster

On reprend la configuration du TP (1 *control-plane* + 2 *workers*). On y ajoute, sur le control-plane, le label `ingress-ready=true` et la publication des ports 80/443, qui serviront à l'Ingress de la partie 5 (ces options ne peuvent pas être ajoutées après la création du cluster).

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ cat kind-config.yaml
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
- role: control-plane
  # Partie 5 (Ingress) : label attendu par ingress-nginx + ports 80/443 publ
iés sur localhost
  kubeadmConfigPatches:
  - |
    kind: InitConfiguration
    nodeRegistration:
      kubeletExtraArgs:
        node-labels: "ingress-ready=true"
  extraPortMappings:
  - containerPort: 80
    hostPort: 80
    protocol: TCP
  - containerPort: 443
    hostPort: 443
    protocol: TCP
- role: worker
- role: worker
```

FIGURE 1.2 – Le fichier `kind-config.yaml` : 3 nœuds, avec les ports 80 et 443 du control-plane publiés sur `localhost`.

```
kind create cluster --name devops-tp --config kind-config.yaml
```

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kind create cluster --name devo
ps-tp --config kind-config.yaml
Creating cluster "devops-tp" ...
 ✓ Ensuring node image (kindest/node:v1.37.0) 🖼
 ✓ Preparing nodes 📦 📦 📦
 ✓ Writing configuration 📜
 ✓ Starting control-plane 🕹️
 ✓ Installing CNI 🔌
 ✓ Installing StorageClass 💾
 ✓ Joining worker nodes 🚜
Set kubectl context to "kind-devops-tp"
You can now use your cluster with:

kubectl cluster-info --context kind-devops-tp

Have a nice day! 👋
```

FIGURE 1.3 – Création du cluster `devops-tp` : image `kindest/node:v1.37.0`, préparation des 3 nœuds, démarrage du control-plane, installation du réseau (CNI) et ajout des 2 workers. kubectl est configuré sur le contexte `kind-devops-tp`.

1.3 Vérification

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl get nodes
NAME                     STATUS   ROLES           AGE   VERSION
devops-tp-control-plane   Ready    control-plane   53s   v1.37.0
devops-tp-worker          Ready    <none>          38s   v1.37.0
devops-tp-worker2         Ready    <none>          38s   v1.37.0
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ docker ps --format "table {{.Na
mes}}\t{{.Ports}}"
NAMES                     PORTS
devops-tp-worker2
devops-tp-worker
devops-tp-control-plane   0.0.0.0:80->80/tcp, 0.0.0.0:443->443/tcp, 127.0.0.
1:35821->6443/tcp
```

FIGURE 1.4 – Les 3 nœuds sont `Ready` : `devops-tp-control-plane` et les deux workers. `docker ps` montre que chaque nœud est un conteneur Docker ; seul le control-plane publie les ports 80/443 et le port de l'API server (6443).

2

Déploiement progressif

2.1 Le Pod

```
kubectl run nginx-pod --image=nginx:alpine
```

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl run nginx-pod --image=n
ginx:alpine
pod/nginx-pod created
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl wait --for=condition=Re
ady pod/nginx-pod --timeout=120s
pod/nginx-pod condition met
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl get pod nginx-pod -o wi
de
NAME        READY   STATUS    RESTARTS   AGE   IP           NODE
   NOMINATED NODE   READINESS GATES
nginx-pod   1/1     Running   0          38s   10.244.2.2   devops-tp-worker
   <none>           <none>
```

FIGURE 2.1 – Le pod `nginx-pod` (image `nginx:alpine`) est `Running` ; le scheduler l'a placé sur `devops-tp-worker`.

Challenge. On entre dans le pod avec `kubectl exec` pour remplacer la page `index.html` :

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl exec nginx-pod -- cat /
usr/share/nginx/html/index.html | grep '<h1>'
<h1>Welcome to nginx!</h1>
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl exec nginx-pod -- sh -c
 'echo "<h1>Hello DevOps</h1>" > /usr/share/nginx/html/index.html'
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl exec nginx-pod -- wget
-qO- http://localhost
<h1>Hello DevOps</h1>
```

FIGURE 2.2 – Avant : la page contient « Welcome to nginx ! ». On écrit `<h1>Hello DevOps</h1>` dans `/usr/share/nginx/html/index.html` ; nginx renvoie maintenant « Hello DevOps ».

Pour l'afficher dans le navigateur, on redirige un port local vers le pod :

```
kubectl port-forward pod/nginx-pod 8080:80
```

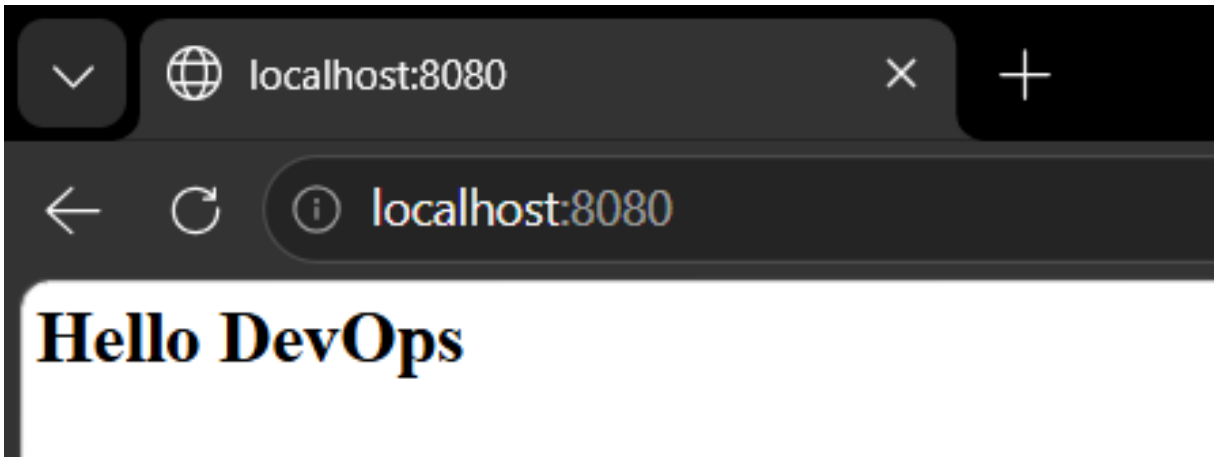


FIGURE 2.3 – La page modifiée « Hello DevOps » ouverte dans Edge via le `port-forward`. (URL : `localhost:8080`)

Remarque : la modification est faite dans le conteneur. Si le pod est recréé, on retrouve la page d'origine : un pod n'est pas fait pour garder des modifications manuelles.

2.2 Le Deployment

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ cat deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web-app
  template:
    metadata:
      labels:
        app: web-app
    spec:
      containers:
      - name: httpd
        image: httpd:2.4
        ports:
        - containerPort: 80
```

FIGURE 2.4 – Le fichier `deployment.yaml` : 3 réplicas, sélecteur `app: web-app`, image `httpd:2.4`.

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl apply -f deployment.yam
l
deployment.apps/web-app created
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl rollout status deployme
nt web-app
Waiting for deployment "web-app" rollout to finish: 0 of 3 updated replicas
are available...
Waiting for deployment "web-app" rollout to finish: 1 of 3 updated replicas
are available...
Waiting for deployment "web-app" rollout to finish: 2 of 3 updated replicas
are available...
deployment "web-app" successfully rolled out
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl get pods -l app=web-app
 -o wide | awk '{print $1, $3, $6, $7}' | column -t
NAME                       STATUS   IP           NODE
web-app-6ff9f45d89-grjgg   Running  10.244.1.2   devops-tp-worker2
web-app-6ff9f45d89-k57fq   Running  10.244.1.3   devops-tp-worker2
web-app-6ff9f45d89-qnswc   Running  10.244.2.3   devops-tp-worker
```

FIGURE 2.5 – `kubectl apply -f deployment.yaml` puis `kubectl get pods -o wide` : les 3 pods sont répartis sur les **deux workers** (2 sur `worker2`, 1 sur `worker`). Aucun pod ne va sur le control-plane, réservé aux composants du cluster (*taint* `NoSchedule`).

Résilience. On supprime un pod à la main :

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl delete pod web-app-6ff9
f45d89-qnswc --wait=false
pod "web-app-6ff9f45d89-qnswc" deleted from default namespace
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl get pods -l app=web-app
 -o wide | awk '{print $1, $3, $5, $7}' | column -t
NAME                       STATUS             AGE  NODE
web-app-6ff9f45d89-grjgg   Running            66s  devops-tp-worker2
web-app-6ff9f45d89-k57fq   Running            66s  devops-tp-worker2
web-app-6ff9f45d89-qnswc   Terminating        66s  devops-tp-worker
web-app-6ff9f45d89-vjm47   ContainerCreating  0s   devops-tp-worker
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ sleep 5; kubectl get pods -l ap
p=web-app -o wide | awk '{print $1, $3, $5, $7}' | column -t
NAME                       STATUS     AGE  NODE
web-app-6ff9f45d89-grjgg   Running    71s  devops-tp-worker2
web-app-6ff9f45d89-k57fq   Running    71s  devops-tp-worker2
web-app-6ff9f45d89-qnswc   Completed  71s  devops-tp-worker
web-app-6ff9f45d89-vjm47   Running    5s   devops-tp-worker
```

FIGURE 2.6 – Le pod `qnswc` est en `Terminating` et un nouveau pod `vjm47` est déjà en `ContainerCreating` (0 s), puis `Running` 5 s plus tard. Le ReplicaSet ramène immédiatement le nombre de pods à l'état désiré (3).

3

Le Networking avec les Services

3.1 ClusterIP

```
kubectl expose deployment web-app --type=ClusterIP --port=80 --target-port=80
```

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl expose deployment web-a
pp --type=ClusterIP --port=80 --target-port=80
service/web-app exposed
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl get svc web-app
NAME      TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)   AGE
web-app   ClusterIP   10.96.185.161   <none>        80/TCP    1s
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl get endpointslices -l k
ubernetes.io/service-name=web-app
NAME            ADDRESSTYPE   PORTS   ENDPOINTS                         AGE
web-app-5ntfp   IPv4          80      10.244.1.2,10.244.1.3,10.244.2.4   1s
```

FIGURE 3.1 – Le service `web-app` reçoit l'IP fixe `10.96.185.161`. Ses *endpoints* sont les IP des 3 pods.

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl run test-pod --image=cu
rlimages/curl --rm -i --restart=Never -- curl -s http://web-app
All commands and output from this session will be recorded in container logs
, including credentials and sensitive information passed through the command
 prompt.
If you don't see a command prompt, try pressing enter.
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html
4/strict.dtd">
<html>
<head>
<title>It works! Apache httpd</title>
</head>
<body>
<p>It works!</p>
</body>
</html>
pod "test-pod" deleted from default namespace
```

FIGURE 3.2 – Depuis un pod temporaire `curlimages/curl`, `curl http://web-app` renvoie la page « It works ! » d'Apache : le service est joignable par son **nom DNS**, sans connaître l'IP des pods.

3.2 NodePort

```
kubectl patch svc web-app -p '{"spec":{"type":"NodePort"}}'
kubectl get svc web-app
```

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl patch svc web-app -p '{
"spec":{"type":"NodePort"}}'
service/web-app patched
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl get svc web-app
NAME      TYPE       CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
web-app   NodePort   10.96.185.161   <none>        80:30657/TCP   43s
```

FIGURE 3.3 – Le service devient `NodePort` : `80:30657/TCP`. Le port **30657** a été choisi automatiquement par Kubernetes dans la plage 30000–32767.

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl get nodes -o custom-col
umns=NAME:.metadata.name,IP:.status.addresses[0].address
NAME                     IP
devops-tp-control-plane  172.19.0.4
devops-tp-worker         172.19.0.3
devops-tp-worker2        172.19.0.2
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ curl -s -m 5 http://172.19.0.3:
30657 || echo "pas de réponse depuis WSL"
pas de réponse depuis WSL
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ docker exec devops-tp-worker cu
rl -s http://localhost:30657 | grep "<title>"
<title>It works! Apache httpd</title>
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ docker exec devops-tp-worker2 c
url -s http://172.19.0.4:30657 | grep "<title>"
<title>It works! Apache httpd</title>
```

FIGURE 3.4 – Le port 30657 répond sur **tous les nœuds**, même sur le control-plane qui n'héberge aucun pod `web-app`. Depuis WSL, il ne répond pas : avec Docker Desktop, le réseau des nœuds (`172.19.0.x`) n'est pas accessible depuis l'hôte. Pour exposer un NodePort sur `localhost` avec kind, il faudrait le déclarer dans `extraPortMappings` à la création du cluster.

3.3 LoadBalancer

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl patch svc web-app -p '{
"spec":{"type":"LoadBalancer"}}'
service/web-app patched
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ sleep 10; kubectl get svc web-a
pp
NAME      TYPE           CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
web-app   LoadBalancer   10.96.185.161   <pending>     80:30657/TCP   105s
```

FIGURE 3.5 – En type `LoadBalancer`, l'`EXTERNAL-IP` reste à `<pending>` : kind n'a pas de fournisseur cloud pour attribuer une IP externe.

Comme le propose le TP, on installe **MetalLB** (v0.16.1) avec un pool d'adresses pris dans le réseau Docker de kind (`172.19.0.0/16`) :

```
apiVersion: metallb.io/v1beta1
kind: IPAddressPool
metadata:
  name: kind-pool
  namespace: metallb-system
spec:
  addresses:
  - 172.19.255.200-172.19.255.250
---
apiVersion: metallb.io/v1beta1
```

```
kind: L2Advertisement
metadata:
  name: kind-l2
  namespace: metallb-system
```

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl apply -f https://raw.gi
thubusercontent.com/metallb/metallb/v0.16.1/config/manifests/metallb-native.
yaml | tail -4
service/metallb-webhook-service created
deployment.apps/controller created
daemonset.apps/speaker created
validatingwebhookconfiguration.admissionregistration.k8s.io/metallb-webhook-
configuration created
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl wait -n metallb-system
--for=condition=Ready pod --all --timeout=180s
pod/controller-76f4cf76b7-dx8gt condition met
pod/speaker-75249 condition met
pod/speaker-bv2z8 condition met
pod/speaker-zc9bs condition met
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl apply -f metallb-pool.y
aml
ipaddresspool.metallb.io/kind-pool created
l2advertisement.metallb.io/kind-l2 created
```

FIGURE 3.6 – Installation de MetalLB : le contrôleur et les 3 *speakers* (un par nœud) sont prêts, puis le pool `kind-pool` est créé.

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ sleep 5; kubectl get svc web-ap
p
NAME      TYPE           CLUSTER-IP      EXTERNAL-IP      PORT(S)        AGE
web-app   LoadBalancer   10.96.185.161   172.19.255.200   80:30657/TCP   4m4
9s
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ docker exec devops-tp-worker cu
rl -s http://172.19.255.200 | grep "<title>"
<title>It works! Apache httpd</title>
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ curl -s -m 5 http://172.19.255.
200 || echo "pas de réponse depuis WSL (Docker Desktop)"
pas de réponse depuis WSL (Docker Desktop)
```

FIGURE 3.7 – Le service reçoit l'`EXTERNAL-IP` **172.19.255.200**, qui répond depuis le réseau des nœuds. Depuis WSL, elle n'est pas joignable, pour la même raison que le NodePort (Docker Desktop).

3.4 Exercice : l'équilibrage de charge

Dans un premier terminal, on suit les logs des pods ; dans un second, on envoie 12 requêtes au service.

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl run load-test --image=c
urlimages/curl --restart=Never -- sh -c 'sleep 2; for i in $(seq 1 12); do c
url -s -o /dev/null -w "requete $i -> %{http_code}\n" http://web-app; done'
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl logs load-test
requete 1 -> 200
requete 2 -> 200
requete 3 -> 200
requete 4 -> 200
requete 5 -> 200
requete 6 -> 200
requete 7 -> 200
requete 8 -> 200
requete 9 -> 200
requete 10 -> 200
requete 11 -> 200
requete 12 -> 200
```

FIGURE 3.8 – Terminal 2 : les 12 requêtes vers `http://web-app` renvoient toutes `200`.

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl logs -l app=web-app -f
--prefix --tail=0 --max-log-requests=10
[pod/web-app-6ff9f45d89-k57fq/httpd] 10.244.1.6 - - [16/Oct/2026:10:42:46 +0
000] "GET / HTTP/1.1" 200 191
[pod/web-app-6ff9f45d89-vjm47/httpd] 10.244.1.6 - - [16/Oct/2026:10:42:46 +0
000] "GET / HTTP/1.1" 200 191
[pod/web-app-6ff9f45d89-grjgg/httpd] 10.244.1.6 - - [16/Oct/2026:10:42:46 +0
000] "GET / HTTP/1.1" 200 191
[pod/web-app-6ff9f45d89-vjm47/httpd] 10.244.1.6 - - [16/Oct/2026:10:42:46 +0
000] "GET / HTTP/1.1" 200 191
[pod/web-app-6ff9f45d89-k57fq/httpd] 10.244.1.6 - - [16/Oct/2026:10:42:46 +0
000] "GET / HTTP/1.1" 200 191
[pod/web-app-6ff9f45d89-vjm47/httpd] 10.244.1.6 - - [16/Oct/2026:10:42:46 +0
000] "GET / HTTP/1.1" 200 191
[pod/web-app-6ff9f45d89-k57fq/httpd] 10.244.1.6 - - [16/Oct/2026:10:42:46 +0
000] "GET / HTTP/1.1" 200 191
[pod/web-app-6ff9f45d89-vjm47/httpd] 10.244.1.6 - - [16/Oct/2026:10:42:46 +0
000] "GET / HTTP/1.1" 200 191
[pod/web-app-6ff9f45d89-k57fq/httpd] 10.244.1.6 - - [16/Oct/2026:10:42:46 +0
000] "GET / HTTP/1.1" 200 191
[pod/web-app-6ff9f45d89-k57fq/httpd] 10.244.1.6 - - [16/Oct/2026:10:42:46 +0
000] "GET / HTTP/1.1" 200 191
[pod/web-app-6ff9f45d89-grjgg/httpd] 10.244.1.6 - - [16/Oct/2026:10:42:46 +0
000] "GET / HTTP/1.1" 200 191
[pod/web-app-6ff9f45d89-k57fq/httpd] 10.244.1.6 - - [16/Oct/2026:10:42:46 +0
000] "GET / HTTP/1.1" 200 191
```

FIGURE 3.9 – Terminal 1 : les 12 requêtes apparaissent dans les logs des **3 pods** (`k57fq` : 6, `vjm47` : 4, `grjgg` : 2).

Analyse : c'est **kube-proxy** qui répartit les connexions. Il programme sur chaque nœud des règles réseau qui envoient chaque nouvelle connexion vers l'un des pods du service, au hasard. Sur un petit nombre de requêtes, la répartition n'est donc pas parfaitement égale, mais tous les pods reçoivent du trafic.

4

Stratégies de déploiement

4.1 Rolling Update (par défaut)

```
kubectl set image deployment/web-app httpd=httpd:latest
kubectl rollout status deployment/web-app
```

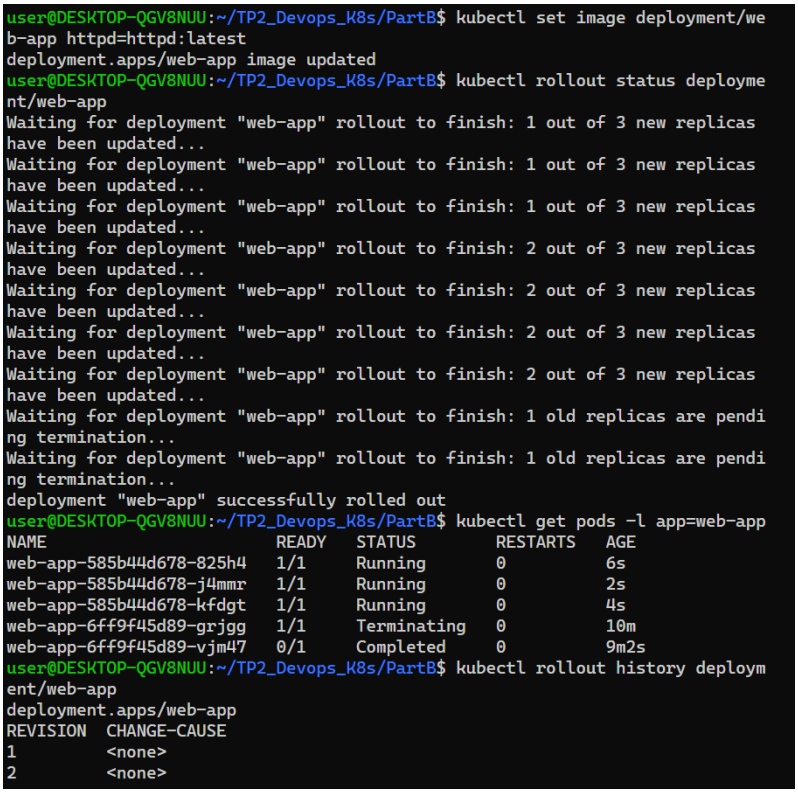


FIGURE 4.1 – Les pods sont remplacés **un par un** (« 1 out of 3 », « 2 out of 3 », « 1 old replicas are pending termination ») : les nouveaux pods (`585b44d678`) démarrent pendant que les anciens tournent encore. L'historique compte 2 révisions.

4.2 Stratégie Recreate

On crée `deployment-recreate.yaml`, une copie de `deployment.yaml` avec :

```
spec:
  strategy:
    type: Recreate
```

Pour mesurer l'impact sur la disponibilité, une « sonde » interroge le service toutes les 0,2 s pendant la mise à jour :

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ cat probe.sh
#!/bin/bash
# Availability probe: query the web-app service every 0.2 s while an update
runs.
# usage: probe.sh <command that triggers the update>
kubectl delete pod probe --ignore-not-found >/dev/null
kubectl run probe --image=curlimages/curl --restart=Never -- \
  sh -c 'for i in $(seq 1 150); do c=$(curl -s -m 1 -o /dev/null -w "%{http_
code}" http://web-app); echo "$(date +%T) $c"; sleep 0.2; done' >/dev/null
kubectl wait --for=condition=Ready pod/probe --timeout=60s >/dev/null
sleep 3
eval "$1"
kubectl rollout status deployment/web-app
kubectl wait --for=jsonpath='{.status.phase}'=Succeeded pod/probe --timeout=
120s >/dev/null
kubectl logs probe > /tmp/probe.txt
echo "--- résultat de la sonde ---"
echo "réponses 200 : $(grep -c ' 200$' /tmp/probe.txt)"
echo "échecs (000) : $(grep -c ' 000$' /tmp/probe.txt)"
```

FIGURE 4.2 – Le script `probe.sh` : un pod `curl` envoie 150 requêtes au service pendant que la mise à jour s'exécute, puis on compte les réponses `200` et les échecs (`000`).

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ bash probe.sh "kubectl apply -f
 deployment-recreate.yaml"
deployment.apps/web-app configured
Waiting for deployment "web-app" rollout to finish: 0 out of 3 new replicas
have been updated...
Waiting for deployment "web-app" rollout to finish: 0 out of 3 new replicas
have been updated...
Waiting for deployment "web-app" rollout to finish: 0 out of 3 new replicas
have been updated...
Waiting for deployment "web-app" rollout to finish: 0 out of 3 new replicas
have been updated...
Waiting for deployment "web-app" rollout to finish: 0 out of 3 new replicas
have been updated...
Waiting for deployment "web-app" rollout to finish: 0 of 3 updated replicas
are available...
Waiting for deployment "web-app" rollout to finish: 1 of 3 updated replicas
are available...
Waiting for deployment "web-app" rollout to finish: 2 of 3 updated replicas
are available...
deployment "web-app" successfully rolled out
--- résultat de la sonde ---
réponses 200 : 141
échecs (000) : 9
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ grep " 000" /tmp/probe_recreate
.txt
10:45:21 000
10:45:21 000
10:45:21 000
10:45:22 000
10:45:22 000
10:45:23 000
10:45:23 000
10:45:24 000
10:45:24 000
```

FIGURE 4.3 – Avec **Recreate** : « 0 out of 3 new replicas » pendant plusieurs secondes, car tous les anciens pods sont arrêtés **avant** la création des nouveaux. La sonde compte **9 échecs consécutifs** (de `10:45:21` à `10:45:24`) : l'application est **totalement indisponible** pendant environ 3 secondes.

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl apply -f deployment.yam
l
deployment.apps/web-app configured
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl get deploy web-app -o j
sonpath='{.spec.strategy.type}'; echo
RollingUpdate
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ bash probe.sh "kubectl set imag
e deployment/web-app httpd=httpd:latest"
deployment.apps/web-app image updated
Waiting for deployment "web-app" rollout to finish: 1 out of 3 new replicas
have been updated...
Waiting for deployment "web-app" rollout to finish: 1 out of 3 new replicas
have been updated...
Waiting for deployment "web-app" rollout to finish: 1 out of 3 new replicas
have been updated...
Waiting for deployment "web-app" rollout to finish: 2 out of 3 new replicas
have been updated...
Waiting for deployment "web-app" rollout to finish: 2 out of 3 new replicas
have been updated...
Waiting for deployment "web-app" rollout to finish: 2 out of 3 new replicas
have been updated...
Waiting for deployment "web-app" rollout to finish: 1 old replicas are pendi
ng termination...
Waiting for deployment "web-app" rollout to finish: 1 old replicas are pendi
ng termination...
Waiting for deployment "web-app" rollout to finish: 1 old replicas are pendi
ng termination...
deployment "web-app" successfully rolled out
--- résultat de la sonde ---
réponses 200 : 146
échecs (000) : 4
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ grep " 000" /tmp/probe_rolling.
txt
10:46:09 000
10:46:09 000
10:46:10 000
10:46:10 000
```

FIGURE 4.4 – Même mesure avec **RollingUpdate** (retour à `deployment.yaml`, puis passage à `httpd:latest`) : seulement **4 échecs isolés**, avec des réponses réussies entre eux. Le service n'est jamais complètement coupé.

Réponse à la question : avec **Recreate**, l'application est **indisponible** pendant la mise à jour (le temps d'arrêter les anciens pods et de démarrer les nouveaux). Elle est utile quand deux versions ne peuvent pas tourner en même temps (par exemple, un changement de schéma de base de données). Avec **RollingUpdate**, il y a toujours des pods disponibles. Les quelques erreurs observées viennent de requêtes envoyées à un pod en cours d'arrêt : on les supprimerait avec une `readinessProbe` et un délai d'arrêt (`preStop`).

TABLEAU 4.1 – Comparaison des stratégies de déploiement (mesures de la sonde)

Stratégie	**Fonctionnement**	**Échecs mesurés (150 requêtes)**	**Disponibilité**
RollingUpdate	Remplacement progressif des pods	4, isolés	Service maintenu
Recreate	Arrêt de tous les pods, puis recréation	9, consécutifs (environ 3 s)	Coupure totale

4.3 Simulation de Canary Release

On garde la v1 (3 réplicas) et on ajoute un déploiement `web-app-v2` à **1 réplica** qui porte le **même label** `app: web-app` : le service `web-app` lui envoie donc aussi du trafic. Pour reconnaître la v2, sa page affiche « v2 canary ».

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app-v2
spec:
  replicas: 1
  selector:
    matchLabels:
      app: web-app
      version: v2
  template:
    metadata:
      labels:
        app: web-app
        version: v2
    spec:
      containers:
      - name: httpd
        image: httpd:2.4
        command: ["sh", "-c", "echo '<h1>v2 canary</h1>' > /usr/local/apache2/htdocs/
    ↪ index.html && httpd-foreground"]
        ports:
        - containerPort: 80
```

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl apply -f deployment-v2-
canary.yaml
deployment.apps/web-app-v2 created
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl rollout status deployme
nt/web-app-v2
Waiting for deployment "web-app-v2" rollout to finish: 0 of 1 updated replic
as are available...
deployment "web-app-v2" successfully rolled out
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl get pods -l app=web-app
 -L version
NAME                          READY   STATUS    RESTARTS   AGE   VERSION
web-app-585b44d678-4cl6s      1/1     Running   0          79s
web-app-585b44d678-9dlbl      1/1     Running   0          78s
web-app-585b44d678-cbznz      1/1     Running   0          76s
web-app-v2-6665ccbf77-kjv6t   1/1     Running   0          2s    v2
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl get endpointslices -l k
ubernetes.io/service-name=web-app
NAME            ADDRESSTYPE   PORTS   ENDPOINTS
          AGE
web-app-5ntfp   IPv4          80      10.244.2.9,10.244.1.13,10.244.2.10 + 1
 more...   11m
```

FIGURE 4.5 – 4 pods portent le label `app: web-app` (3 v1 + 1 v2, colonne `VERSION`) : le service a maintenant 4 endpoints.

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ cat canary-test.sh
#!/bin/bash
# Canary test: 100 requests to the web-app service, count v1 / v2 answers.
kubectl delete pod canary-test --ignore-not-found >/dev/null
kubectl run canary-test --image=curlimages/curl --restart=Never -- \
  sh -c 'for i in $(seq 1 100); do curl -s http://web-app; echo; done' >/dev
/null
kubectl wait --for=jsonpath='{.status.phase}'=Succeeded pod/canary-test --ti
meout=120s >/dev/null
kubectl logs canary-test > /tmp/canary.txt
echo "v1 (It works!) : $(grep -c '<title>It works' /tmp/canary.txt)"
echo "v2 (canary)    : $(grep -c 'v2 canary' /tmp/canary.txt)"
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ bash canary-test.sh
v1 (It works!) : 77
v2 (canary)    : 23
```

FIGURE 4.6 – Sur 100 requêtes, **77** vont à la v1 et **23** à la v2, soit environ **25 %** : 1 pod sur 4.

Remarque : la part de trafic de la canary dépend ici du **nombre de pods**. Pour un contrôle plus fin (par exemple 5 %), on utilise un Ingress ou un service mesh.

5

Application multi-conteneurs avec ClusterIP et Ingress

5.1 Contrôleur Ingress

Un objet Ingress a besoin d'un **contrôleur**. On installe **ingress-nginx** (v1.15.1), la solution documentée pour kind.

> Remarque : le projet ingress-nginx est **archivé depuis mars 2026** (plus de mises à jour). Il reste utilisable pour un TP, mais en production on choisirait aujourd'hui un autre contrôleur (Traefik…) ou la **Gateway API**.

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl apply -f https://raw.gi
thubusercontent.com/kubernetes/ingress-nginx/controller-v1.15.1/deploy/stati
c/provider/kind/deploy.yaml | tail -3
job.batch/ingress-nginx-admission-patch created
ingressclass.networking.k8s.io/nginx created
validatingwebhookconfiguration.admissionregistration.k8s.io/ingress-nginx-ad
mission created
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl wait -n ingress-nginx -
-for=condition=Ready pod -l app.kubernetes.io/component=controller --timeout
=240s
pod/ingress-nginx-controller-596f5b6bcf-7z5hf condition met
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl get pods -n ingress-ngi
nx -o wide | awk "{print \$1, \$3, \$7}" | column -t
NAME                                       STATUS   NODE
ingress-nginx-controller-596f5b6bcf-7z5hf  Running  devops-tp-worker2
```

FIGURE 5.1 – Installation d'ingress-nginx : le contrôleur est prêt, mais il a été placé sur `devops-tp-worker2`.

Or seul le control-plane publie le port 80 sur `localhost`. Cette version du manifeste ne force plus le placement sur le nœud `ingress-ready=true`, donc on l'ajoute :

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl patch deployment ingres
s-nginx-controller -n ingress-nginx -p '{"spec":{"template":{"spec":{"nodeSe
lector":{"ingress-ready":"true"}}}}}'
deployment.apps/ingress-nginx-controller patched
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl rollout status deployme
nt ingress-nginx-controller -n ingress-nginx
Waiting for deployment "ingress-nginx-controller" rollout to finish: 0 of 1
updated replicas are available...
deployment "ingress-nginx-controller" successfully rolled out
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl get pods -n ingress-ngi
nx -l app.kubernetes.io/component=controller -o wide | awk '{print $1, $3, $
7}' | column -t
NAME                                       STATUS   NODE
ingress-nginx-controller-575c4d8bc6-sdhdk  Running  devops-tp-control-plane
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ curl -s -o /dev/null -w "localh
ost:80 -> %{http_code}\n" http://localhost/
localhost:80 -> 404
```

FIGURE 5.2 – Après le `patch` (`nodeSelector: ingress-ready=true`), le contrôleur tourne sur `devops-tp-control-plane` et `localhost:80` répond `404` : le contrôleur est joignable mais aucune règle Ingress n'existe encore.

5.2 Application multi-conteneurs

Chaque pod contient **deux conteneurs** qui partagent un volume `emptyDir` :

— `web` (`nginx:alpine`) sert la page `index.html` ;

— `content-writer` (`busybox`), un *sidecar*, réécrit cette page toutes les 5 secondes avec le nom du pod et l'heure.

Le service `multi-app` est de type **ClusterIP** (interne) et l'**Ingress** envoie les requêtes `http://localhost/` vers ce service.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: multi-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: multi-app
  template:
    metadata:
      labels:
        app: multi-app
    spec:
      volumes:
      - name: html
        emptyDir: {}
      containers:
      - name: web
        image: nginx:alpine
        ports:
        - containerPort: 80
        volumeMounts:
        - name: html
          mountPath: /usr/share/nginx/html
      - name: content-writer
        image: busybox:1.37
        command: ["sh", "-c"]
        args:
        - |
          while true; do
            echo "<h1>Hello DevOps - application multi-conteneurs</h1><p>Pod : $(
    ↪ hostname)</p><p>Mis a jour par le sidecar : $(date)</p>" > /html/index.html
            sleep 5
          done
        volumeMounts:
        - name: html
          mountPath: /html
---
```

```
apiVersion: v1
kind: Service
metadata:
  name: multi-app
spec:
  type: ClusterIP
  selector:
    app: multi-app
  ports:
  - port: 80
    targetPort: 80
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: multi-app
spec:
  ingressClassName: nginx
  rules:
  - host: localhost
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: multi-app
            port:
              number: 80
```

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl apply -f multi-app.yaml
deployment.apps/multi-app created
service/multi-app created
ingress.networking.k8s.io/multi-app created
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl rollout status deployme
nt multi-app
Waiting for deployment "multi-app" rollout to finish: 0 of 2 updated replica
s are available...
Waiting for deployment "multi-app" rollout to finish: 1 of 2 updated replica
s are available...
deployment "multi-app" successfully rolled out
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl get pods -l app=multi-a
pp -o wide | awk "{print \$1, \$2, \$3, \$7}" | column -t
NAME                         READY  STATUS   NODE
multi-app-6d85958bcf-l6x8c  2/2    Running  devops-tp-worker2
multi-app-6d85958bcf-xfb2b  2/2    Running  devops-tp-worker
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl get svc multi-app
NAME        TYPE        CLUSTER-IP    EXTERNAL-IP   PORT(S)   AGE
multi-app   ClusterIP   10.96.97.64   <none>        80/TCP    23s
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl get ingress multi-app
NAME        CLASS   HOSTS       ADDRESS   PORTS   AGE
multi-app   nginx   localhost             80      23s
```

FIGURE 5.3 – Les 2 pods `multi-app` sont `2/2` (deux conteneurs prêts dans chaque pod), un sur chaque worker. Le service `multi-app` est un ClusterIP et l'Ingress `multi-app` (classe `nginx`) répond pour l'hôte `localhost`.

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl get pod multi-app-6d859
58bcf-l6x8c -o jsonpath='{.spec.containers[*].name}'; echo
web content-writer
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl get ingress multi-app
NAME        CLASS   HOSTS       ADDRESS   PORTS   AGE
multi-app   nginx   localhost             80      33s
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ curl -s http://localhost/; echo
<h1>Hello DevOps - application multi-conteneurs</h1><p>Pod : multi-app-6d859
58bcf-xfb2b</p><p>Mis a jour par le sidecar : Fri Oct 16 10:53:20 UTC 2026</
p>
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ sleep 6; curl -s http://localho
st/; echo
<h1>Hello DevOps - application multi-conteneurs</h1><p>Pod : multi-app-6d859
58bcf-l6x8c</p><p>Mis a jour par le sidecar : Fri Oct 16 10:53:28 UTC 2026</
p>
```

FIGURE 5.4 – Le pod contient bien les conteneurs `web` et `content-writer`. `curl http://localhost/` passe par l'Ingress et renvoie la page écrite par le sidecar ; deux requêtes successives sont servies par deux pods différents, avec une heure différente.



FIGURE 5.5 – La page dans Edge, servie par le pod `xfb2b` (mise à jour à `11:01:49`). (URL : `localhost`)



FIGURE 5.6 – Après rafraîchissement, la page est servie par l'autre pod (`l6x8c`) et l'heure a changé (`11:02:47`). (URL : `localhost`)

6

Questions de validation

6.1 Troubleshooting : ImagePullBackOff

Un pod est en `ImagePullBackOff` quand le kubelet n'arrive pas à télécharger l'image. Les trois causes principales sont :

1. **Nom ou tag d'image incorrect** : l'image ou la version n'existe pas dans le registre (faute de frappe, tag supprimé).
2. **Registre privé sans authentification** : l'image demande des identifiants, mais aucun `imagePullSecret` n'est configuré (ou il est invalide).
3. **Problème réseau ou de registre** : le nœud ne peut pas joindre le registre (pas d'accès Internet, DNS, proxy, pare-feu) ou le registre limite les téléchargements (*rate limit* de Docker Hub).

```
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl run bad-image --image=n
ginx:version-inexistante
pod/bad-image created
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ sleep 25; kubectl get pod bad-i
mage
NAME        READY   STATUS             RESTARTS   AGE
bad-image   0/1     ImagePullBackOff   0          25s
user@DESKTOP-QGV8NUU:~/TP2_Devops_K8s/PartB$ kubectl describe pod bad-image
| grep -E 'Failed|BackOff' | cut -c1-200 | head -4
      Reason:       ImagePullBackOff
  Normal   BackOff    24s                kubelet            spec.containers{b
ad-image}: Back-off pulling image "nginx:version-inexistante"
  Warning  Failed     24s                kubelet            spec.containers{b
ad-image}: Error: ImagePullBackOff
  Warning  Failed     8s (x2 over 24s)   kubelet            spec.containers{b
ad-image}: Failed to pull image "nginx:version-inexistante": rpc error: code
 = NotFound desc = failed to pull and unpack ima
```

FIGURE 6.1 – Exemple de la cause 1 : le pod `bad-image` utilise le tag `nginx:version-inexistante`. Il passe en `ImagePullBackOff` et `kubectl describe` affiche l'erreur `NotFound` : l'image n'existe pas. On diagnostique toujours avec `kubectl describe pod <pod>` (section *Events*).

6.2 Réseau : accès depuis le navigateur via localhost

Avec kind, les nœuds sont des conteneurs Docker et leur réseau n'est pas accessible depuis l'hôte. On a utilisé les deux méthodes proposées :

— **`kubectl port-forward`** (partie 2.1) : `kubectl port-forward pod/nginx-pod 8080:80` puis `http://localhost:8080`. C'est simple, mais temporaire : la redirection ne fonctionne que tant que la commande tourne.

— **Ingress** (partie 5) : le cluster est créé avec `extraPortMappings` (port 80 du control-plane publié sur l'hôte) et un contrôleur Ingress. On accède ensuite à `http://localhost/` sans commande à laisser ouverte.

6.3 Clean-up

Toute l'infrastructure (les 3 nœuds et tout ce qu'ils contiennent) se supprime en une seule commande :

```
kind delete cluster --name devops-tp
```

Conclusion générale

Cette partie B a permis de créer un cluster kind à 3 nœuds et d'y manipuler les objets de base de Kubernetes : pods, déploiements avec auto-réparation, services ClusterIP, NodePort et LoadBalancer (avec MetalLB). On a aussi mesuré l'effet des stratégies de déploiement sur la disponibilité (Recreate : coupure d'environ 3 s ; RollingUpdate : service maintenu), réalisé une canary à environ 25 %, et exposé une application multi-conteneurs (avec un sidecar) par un service ClusterIP et un Ingress.